

A Time Interval Based Consistency Control Algorithm for Interactive Groupware Applications

Rui Li and Du Li

Department of Computer Science
Texas A&M University
College Station, Texas 77843-3112 USA

Chengzheng Sun

School of Computing and Information Technology
Griffith University
Brisbane, Qld 4111, Australia

Abstract—Traditional concurrency control methods such as locking and serialization are not suitable for distributed interactive applications that demand fast local response. Operational transformation (OT) is the standard solution to concurrency control and consistency maintenance in group editors, an important class of interactive groupware applications. It generally trades consistency for local responsiveness, because human users can often tolerate temporary inconsistencies but do not like their interactions be lost or nondeterministically blocked. This paper presents a time interval based operational transformation algorithm (TIBOT) that overcomes the various limitations of previous related work. Our approach guarantees content convergence and is significantly more simple and efficient than existing approaches. This is achieved in a pure replicated architecture by using a linear clock and by posing some constraints on communication that are reasonable for the application domain.

I. INTRODUCTION

The fast growth of the Internet has opened a new era of distributed computing. A notable application domain is computer-supported cooperative work (CSCW), which allows a group of geographically dispersed users to perform a common task across the computer network through a virtual user interface. In the CSCW area, group editors are often used as an effective vehicle to investigate a range of research issues that are widely present in many CSCW applications. With group editors, distributed users can edit a shared document at the same time. Typical examples of group editors include shared whiteboards, group authoring and documentation, collaborative diagramming, and concurrent software engineering. In fact, a range of distributed interactive applications (e.g., distributed virtual environments and network games) can be modeled as group editors due to their coordinated manipulation of shared objects.

The interactive nature of such applications demand that the effect of a user's action be seen by the local user and other users in a timely manner. This requirement is particularly difficult to achieve in wide-area and mobile wireless networks, where high communication latencies are common. Therefore, as a design convention, interactive groupware applications such as group editors usually adopt a replicated architecture. In a typical group editor (e.g., Grove [4] and Reduce [16]), the shared document is replicated at each site. A user's editing operations are executed immediately on the local replica for quick response and then propagated to other sites for synchronization.

However, replication leads to the problem of replica consistency due to the possibility of different ordering of operations

at different sites as a result of arbitrary communication. Concurrency control methods that work effectively in traditional distributed computing systems [3], such as locking and serialization, have been found not suitable for interactive groupware applications [4] [5]. In general, optimistic (locking or serialization) methods allow inconsistencies to happen and seek to repair them afterwards in an efficient way. Thus they are able to achieve high local response but may frequently cause the loss of users' work due to the failure to acquire a lock or the rollback of operations that have been executed out of order. Pessimistic methods, on the other hand, prevent inconsistencies from happening by taking conservative measures to avoid simultaneous conflicting operations. Consistency is often achieved at the price of local response. Interactive groupware applications call for concurrency control methods that are able to achieve high local response without losing users' interaction results when conflicts occur.

The group editing community has contributed a family of concurrency control algorithms, called operational transformation (OT) [4] [15]. As an optimistic approach, OT algorithms replicate documents and allow local operations to be executed immediately for fast response. Remote operations that are concurrent to locally executed operations are **transformed** before execution. In this way, both local and remote editing results are preserved without loss. Sun et al. [16] further demonstrates that OT is potentially able to simultaneously achieve the following three properties that are desired in interactive groupware applications: convergence, causality preservation, and intention preservation. As a result, OT algorithms are widely used in group editors, to name but a few, [2] [4] [6] [10] [12] [16] [14] [17].

Unfortunately, existing OT algorithms are not without problems. In this paper, we are particularly interested in two OT puzzles that existing algorithms are not able to solve efficiently. Basic OT concepts and these two puzzles will be introduced in Section II. We will then present a novel approach to solving these two puzzles, which consists of a time interval based interaction model (Section III) and an OT algorithm based on this model (Section IV). Our approach significantly simplifies existing OT algorithms and is thus more efficient. The advantages of our approach over previous work is argued in Section V. Section VI gives a correctness proof of our approach. Section VII summarizes the technical contributions of this paper and points out future directions of research.

II. OPERATIONAL TRANSFORMATION

A family of OT algorithms have been evolved out of the seminal dOPT algorithm by Ellis and Gibbs [4], including Jupiter [10], adOPTed [12], GOT [16], and GOTO [15]. As a convention, the shared document is usually modeled as a linear string and editing operations are modeled by two primitives, insert and delete. For example, a document's initial state is represented by a string "abcd". Based on this state performing operation $\text{Ins}(0, "x")$ inserts the character 'x' right **before** the current character at position 0, which results in a new document state "xabcd". Operation $\text{Del}(1, "b")$ deletes the character 'b' **at** position 1 and changes the document state to "acd". Intuitively, transformation shifts the position parameter of an insert or delete operation to incorporate effects of preceding operations, as will be evident in later examples. In this paper we use the standard concurrent relation $\parallel$ and happened-before (or causal ordering) relation $\rightarrow$ of operations as defined in [7] without further definitions.

A. Inclusion and exclusion transformation

Sun et al [16] distinguishes inclusion transformation (IT) and exclusion transformation (ET) functions. The IT function requires that if O_1 is transformed against O_2 , the effect of executing its transformed version O'_1 ($O'_1 = IT(O_1, O_2)$) on the document state that contains the effect of O_2 should be the same as the effect of executing O_1 on the document state that does not contain the effect of O_2 . The ET function means that O_1 is transformed against O_2 ($O'_1 = ET(O_1, O_2)$) in such a way that the effect of O_2 is effectively excluded from O_1 .

B. Transformation properties

Given two operations O_1 and O_2 , let $O'_1 = IT(O_1, O_2)$ and $O'_2 = IT(O_2, O_1)$, transformation function IT is required to possess the following two properties [12]:

$$\text{TP1: } O_1 \circ O'_2 \equiv O_2 \circ O'_1$$

$$\text{TP2: } IT(IT(O, O_1), O'_2) = IT(IT(O, O_2), O'_1)$$

TP1 ensures that if $O_1 \parallel O_2$, the effect of executing O_1 before O_2 is the same as executing O_2 before O_1 . TP2 ensures that transforming any operation O along different paths will yield the same result. These two properties actually ensure that arbitrary communication order can lead to a consistent final document state. In other words, it is not necessary to conservatively enforce a global total order of operations because inconsistencies can always be repaired with operational transformation, if TP1 and TP2 can be satisfied.

C. The dOPT puzzle

The original dOPT algorithm failed in some cases. These cases were re-discovered by several research groups [2] [10] [12] [16] and are termed as the dOPT puzzle.

Figure 1 illustrates a simple scenario of the dOPT puzzle. Consider two users, A and B, start a group editing session from the same document state "abcd". At site 1, user A executes operation $O_1 = \text{Del}(0, "a")$ and then $O_2 = \text{Ins}(2, "x")$, yielding

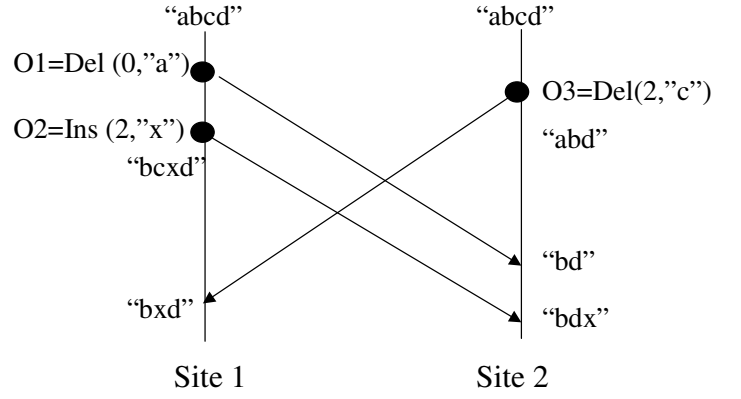


Fig. 1. A scenario of the dOPT puzzle.

document state "bcxd". At site 2, user B concurrently performs operation $O_3 = \text{Del}(2, "c")$, yielding "abd".

According to the dOPT algorithm, a remote operation must be transformed against concurrent operations that have been executed locally. Then the transformed form is executed. So when site 1 receives O_3 from site 2, O_3 is transformed against O_1 and O_2 in order and becomes $O'_3 = \text{Del}(1, "c")$. After O'_3 is executed, the document state at site 1 becomes "bxd".

At site 2, when O_1 arrives, it is transformed against O_3 . O'_1 is exactly the same as O_1 because its operation position precedes that of O_3 . The document state of site 2 becomes "bd" after executing O'_1 . And then O_2 arrives and is transformed against O_3 , which results in $O'_2 = O_2$ because of the transformation rule (insert against delete) [4]. After the execution of O'_2 , the document state of site 2 becomes "bdx", which is not consistent with the state of site 1.

The root of the dOPT puzzle is that, when operation O_2 is transformed against O_3 at site 2, these two concurrent operations to be transformed are not originated from the same document state. The document state from which O_2 was generated includes the effect of O_1 , while the document state from which O_3 was generated does not. So the position parameters of O_2 and O_3 are not comparable.

D. The TP2 puzzle

In addition to the dOPT puzzle, another transformation puzzle was discovered by Sun et al. [16], which is illustrated by the scenario shown in Figure 2.

In figure 2, three users start with the same initial document state "abc". User A performs an insert operation $O_1 = \text{Ins}(2, "1")$ at site 1 and its document state becomes "ab1c". User B performs $O_2 = \text{Ins}(1, "2")$ at site 2 and its document state becomes "a2bc". At site 3, user C performs $O_3 = \text{Del}(1, "b")$ and the document state becomes "ac". Those three operations are mutually concurrent with each other.

When site 2 receives O_3 , it is transformed against O_2 and becomes $O'_3 = \text{Del}(2, "b")$. After O'_3 is executed, the document state of site 2 becomes "a2c". When O_1 arrives at site 2, it is transformed against O_2 and O'_3 in a sequel and becomes $O'_1 = \text{Ins}(2, "1")$, which is the same as O_1 . After executing O'_1 , the document state of site 2 becomes "a21c".

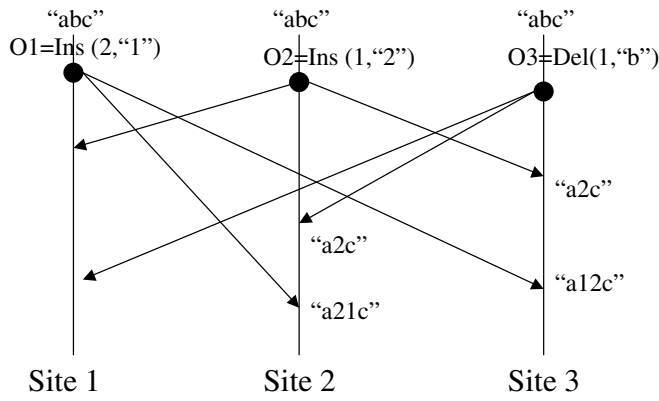


Fig. 2. The scenario of the TP2 puzzle.

At site 3, O_2 arrives first and is transformed against O_3 . The transformation result O_2' is the same as O_2 and the document state becomes "a2c". When O_1 arrives, it is transformed against O_3 first and changed to $O_1' = \text{Ins}(1, "1")$. O_1' is transformed against $O_2' = \text{Ins}(1, "2")$. Since the site id of O_2' is greater than one of O_1' , $O_1' = \text{Ins}(1, "1")$ is not modified. After O_1' is executed, the document state of site 3 becomes "a12c", which is not consistent with the state of site 2.

The puzzle seems to be fixable by simply applying the following rule as proposed in [12]: when two insert operations have the same position parameter, the position of the operation with a larger site identifier will be shifted. Unfortunately, this quick fix works only in this case but fails in another similar scenario obtained by simply reversing the site identifiers of O_1 and O_2 . The root of the problem is deeper than it appears and requires a more sophisticated solution than just considering the site identifier. The puzzle has a different nature from the dOPT puzzle. Since TP2 fails in this case, we call it TP2 puzzle.

III. THE INTERACTION MODEL

In this section, we present a novel interaction model that is based on time intervals. We first introduce some necessary concepts, define operation propagation and synchronization rules, and then study some interesting properties of our model.

A. Time intervals based on logical clocks

Most known existing concurrency control algorithms for real-time group editors, including dOPT, adOPTed, and GOTO, used state vectors (or vector timestamps) to detect precedence and concurrent relationships between operations [4][15]. In their interaction model, local operations are always executed immediately once generated and then propagated to other sites. The synchronization of remote operations are delayed only when they are not causally ready. The propagation of operations may be delayed to save network bandwidth [9] but the delay is not a necessary part of the model. Unlike previous work, our model is based on time intervals, the advantages of which will become clear later in this paper.

In this model, every site maintains a linear logical clock. All clocks are initialized to a common value, e.g., 0. Clocks at different sites may tick at different speeds but must take the

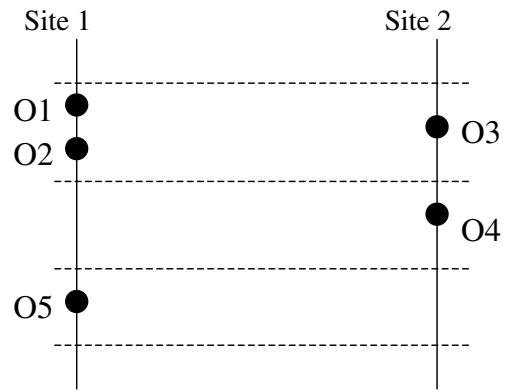


Fig. 3. Operations and time intervals.

same sequence of values, e.g., 0, 1, 2, 3,.... A time interval is the period between two consecutive clock ticks. The length of time intervals at each site is chosen basing on the principle that there are editing operations in as many time intervals as possible. Hence in general logical clocks should tick no faster than the keyboard response time. Further, although time interval lengths can be different at cooperating sites, in this paper we assume they are the same to simplify discussion.

Every operation is timestamped by the current local clock value to indicate which time interval it belongs to. Function $TI(O)$ denotes the time interval of operation O . Two operations, O_i and O_j , are in the same time interval iff $TI(O_i) = TI(O_j)$, no matter whether $i = j$ or not. A new time interval at a site is announced by broadcasting a control message that carries the clock value of the new time interval to all remote sites. This control message can be piggybacked in messages carrying editing operations.

Figure 3 shows an example. At site 1, O_1 and O_2 are generated in the first time interval. $TI(O_1)$ and $TI(O_2)$ are assigned 1, respectively. At site 2, O_3 is generated in the first time interval as well. Hence $TI(O_3)$ is also assigned 1. Similarly, O_4 at site 2 is generated in the second time interval and $TI(O_4)$ is assigned 2. At site 1, O_5 is generated in the third interval. $TI(O_5)$ is assigned 3 although no operation is generated in the second interval.

B. Total ordering and operation context

Definition 1: Total ordering relation " $\Rightarrow$ ": Given two operations O_1 and O_2 , generated at sites i and j , respectively, then $O_1 \Rightarrow O_2$, iff: (1) $TI(O_1) < TI(O_2)$, or (2) $TI(O_1) = TI(O_2)$ and $i < j$, or (3) $TI(O_1) = TI(O_2)$, $i = j$ and $O_1 \rightarrow O_2$.

It is easy to see that the total ordering relation " $\Rightarrow$ " is consistent with the precedence relation " $\rightarrow$ " in the sense that if $O_1 \rightarrow O_2$, then $O_1 \Rightarrow O_2$. In addition, we use a linear history buffer to store the executed operations at each site. In our model, we ensure that the operations are stored in the total order defined above in the history buffer at each site. However, operations are allowed to be executed in any order as long as their causality is preserved (and the following synchronization rules are observed).

To capture the relation among operations for correct transformation, here we introduce the notion of **operation context**. The context of an operation is the sequence of operations that have been executed on the initial document state up to the current document state. Based on the discussion in Section II, we know that an operation may have many different execution forms, after it is transformed against different sequences of operations. Every execution form of an operation is based on a certain context. So we define the document state from which an execution form of an operation is originated as the context of the execution form of that operation. Note in later discussions the term of operation refers to one of the execution forms of that operation. For any given two operations, O_1 and O_2 , if their execution contexts are the same, these two operations are **contextually equivalent**. If O_1 is executed consecutively before O_2 at one site, then these two operations are **contextually serialized**.

C. Propagation and synchronization rules

In order to constrain the complexity of the consistency problem and develop more efficient concurrency control algorithms, we introduce the following operation propagation and synchronization rules.

Propagation rule 1: *A local operation is propagated only when the current time interval is over and the operation has been transformed with all operations that have been executed in earlier time intervals.*

In the previous interaction model of realtime group editors, a local operation can be propagated to remote sites immediately as soon as it is generated. But in our model, a local operation always needs to be transformed against all operations generated in previous time intervals before it can be propagated. In other words, an operation that will be propagated must include the effects of all operations that have happened in previous time intervals. Hence operation propagation is normally delayed. Note that operations in the first time interval are always propagated as soon as the interval is over because no operation happened before them.

Propagation rule 2: *When a time interval is over, if no operation has been generated locally, a message is propagated to notify other sites that there was no operation generated in the past time interval. This message is called the zero-operation message.*

This rule ensures that system interaction is not blocked for long time because, in our model, if a site can not receive all operations that happened in the previous time intervals, it can not send new messages to other sites. As a special synchronization mechanism, zero-operation messages can be used for detecting the liveness of cooperating sites.

If time intervals are short, fine-grained real-time collaboration is achieved. However, network traffic is increased due to the increase in the number of propagation messages. On the other hand, if the time intervals are set longer, network overhead can be decreased at the price of reduced mutual awareness between collaborators. To satisfy the requirements of a range of collaborative applications, the time interval

should be implemented as a system parameter that can be adjusted by the users.

Synchronization rule 1: *Any remote operation received during a time interval can be synchronized only when (1) local time interval is over, if the time interval of this operation is the same as the current local time interval; and (2) all operations totally preceding this operation have been executed.*

Any local operations cannot be interrupted by any remote operations carrying the same timestamp. Thus this rule ensures that the first operation in the same time interval from different sites must be context equivalent and concurrent. When a local operation is propagated, it only includes the effects of operations which have been executed in previous time intervals, not including the effects of remote operations that were generated in the same time interval.

Synchronization rule 2: *When a remote operation is to be executed, all operations that have been executed but out of the total order must be first undone, then preform this operation, and then redo these undone operations in sequence. These operations to be done or redone need to be transformed before they are executed.*

This rule actually explains the so-called undo/do/redo scheme [16]. One assumption made by this rule is that all operations in the group editor are reversible. It is required that buffered operations should contain enough information in order to be undone and redone. It should be noted that the undo operation involved in the undo/do/redo scheme is only an internal operation, rather than an external operation initiated from the user interface [6]. Therefore, the undo/do/redo scheme should be implemented in such a way that only the final result (instead of intermediate ones) produced at the end of the undo/do/redo process is reflected on the user interface.

Synchronization rule 3: *Operations with the same timestamp and site id are always synchronized as an entirety.*

The rule ensures that remote operations with the same timestamp and site id are processed as an entirety in transformations against local operations. So we introduce a new concept, *group operation*, to denote them. We define $TI(GO) = TI(O_i)$ where O_i is an operation in GO. Since these operations are contextually serialized operations, we define that the context of a GO is the same as the context of the earliest operation generated during the time interval of GO. If a GO has the same context as an operation or another GO, we say they are contextually equivalent. A singular operation can be considered as a special-case, fine-grained group operation. In remote operations synchronization, we always use GO as basic unit of remote operations.

D. Interesting properties

Compared with existing consistency models for group editors, e.g., [12] [16] [15], the above rules reduce the complexity of communication and interaction. Here we examine some interesting properties derived from them.

Property 1: If GO_r is a remote group operation and O is a locally executed operation and $TI(O) < TI(GO_r)$, then GO_r does not need to be transformed against O .

Property 1 is based on the propagation rules, which ensure that any operation must be transformed against all operations carrying earlier timestamps before it is propagated. So when this operation is synchronized at a remote site, it only needs to be transformed against all executed operations carrying the same or later timestamps at that site.

Property 2: Assume that GO_r is a synchronization-ready remote group operation and O is a locally executed operation. If $TI(O) \geq TI(GO_r)$, all operations in GO_r must be concurrent with O .

Due to propagation and synchronization rules, GO_r only includes the effect of all operations carrying earlier timestamps. That is, GO_r does not include the effect of O . Meantime, since GO_r is not synchronized yet at that moment, O does not include the effect of any operations in GO_r . Due to the definition of the concurrent relation, all operations in GO_r are concurrent with O . This property ensures that the concurrent relation between operations can be detected correctly in our model. So we do not need to use a vector clock.

Property 3: Assume that GO_r is a synchronization-ready remote group operation and the local history buffer $HB = [O_1, \dots, O_n]$. If O_i is the first operation in HB such that $TI(O_i) \geq TI(GO_r)$, then operations in GO_r are concurrent with $[O_i, \dots, O_n]$ and GO_r and O_i are context-equivalent.

Due to the definition of history buffer in our model, for any O_j in HB , if $j > i$, we have $TI(O_j) \geq TI(O_i)$. Due to property 2, $[O_i, \dots, O_n]$ are concurrent with operations in GO_r . In addition, the operation synchronization rule ensures that when a remote group operation is synchronized, all operations carrying earlier timestamps have been synchronized. Since $TI(O_i) \geq TI(GO_r)$, O_i includes the effects of all operations carrying earlier timestamps than GO_r . Meantime, based on the definition of history buffer, O_i does not include the effects of any operations carrying equal or later timestamps. According to property 1, GO_r also only includes the effects of all operations carrying earlier timestamps. So GO and O_i are originated from the same state and are context-equivalent.

Property 4: Local operations are always executed immediately once generated.

Since our model does not impose any constraints on the execution of local operations, it ensures that local operations are not blocked so that high local response can be achieved. Note that local operation execution does not need to follow the *undo/do/redo* scheme. When a local operation is executed, although we cannot ensure that all operations totally preceding it have been executed, the synchronization rules ensure that no operation totally following it has been executed. However, when remote operations are synchronized, the total order is maintained by the *undo/do/redo* scheme.

Property 5: Exclusion transformation (ET) [16] is not necessary in our model.

For any O_i and O_j , $ET(O_i, O_j)$ is necessary only when O_j is based on a document context that is earlier than the context of O_i . As an example consider operations O_2 and O_3 in Figure 1. We need to exclude the effect of O_1 from O_2 first so that the position parameters of O_2 and O_3 are based

on the same document state and comparable. This is because no execution form of O_2 based on that document context is stored. Exclusion transformation can be used to compute the required execution form of O_2 [15].

There are two implications in this example: First, ET is not always possible if O_2 is dependent on O_1 , e.g., O_2 deletes part of the characters inserted by O_1 . Hence the use of ET often complicates an OT algorithm and makes it problematic. Second, if the necessary execution form of O_2 is stored in the system, as in [12], then ET is not necessary. However, due to the propagation and synchronization rules defined in our model, we do not need ET either, as proved below.

Proof: Without losing generality, we assume that GO is a synchronization-ready remote operation. At that moment, a local history is $[O_1, O_2, \dots, O_n]$. Assume that O_i is the first operation concurrent with GO in the history buffer. It means that $[O_i, \dots, O_n]$ are concurrent with GO , due to property 3. If there exist $[O_j, O_{j+1}, \dots, O_n]$ which follow GO in the total order, due to undo/do/redo scheme, these operations are undone first. And then do GO after GO is transformed against $[O_i, \dots, O_{j-1}]$. Due to property 3, GO and O_i are context equivalent. So transformation between them can be processed without exclusion transformation, as noted in [13]. Due to the history buffer definition of our model, $[O_j, \dots, O_n]$ are *contextually serialized* operations and O_j only includes the effect of $[O_1, \dots, O_{j-1}]$. So GO' and O_j are context equivalent. So transformations of $[O_j, \dots, O_n]$ against GO' do not need exclusion transformation. After that, $[O_j, \dots, O_n]$ are redone. Therefore we conclude that *exclusion transformation* is not necessary in our model. In fact, at the same time, we also proved that dOPT puzzle is free in our model, because dOPT puzzle comes from transformation between two operations based on different document contexts.

IV. THE TIBOT CONTROL ALGORITHM

Due to the rules we defined and properties derived from these rules, concurrency problem is simplified in our model. In particular, based on the proof of property 5, the concurrency problem in our model can be reduced (from the three cases discussed in [15]) to the following two cases. Assume that O is a execution-ready operation. $[O_1, O_2, \dots, O_n]$ are a sequence of operations that have been executed locally and stored in the local history buffer (HB). For simplicity, suppose $HB = [O_1, O_2, O_3]$.

Case 1: $O_1 \rightarrow O, O_2 \rightarrow O, O_3 \rightarrow O$

All operations in HB are preceding O . Therefore $O' = O$, i.e., no transformation is needed.

Case 2: $O_1 \rightarrow O, O_2 \parallel O, O_3 \parallel O$

Operations preceding O are stored in HB before operations that are independent of O . Then O' is obtained by transforming O against concurrent operations, O_2 and O_3 , in a sequel.

In this section, we give a top-level control algorithm TIBOT that is based on these two cases and the undo/do/redo scheme. TIBOT is called only when a remote group operation is ready for synchronization. In the following, given a group operation $L = [O_1, O_2, \dots, O_i, \dots, O_j, \dots, O_n]$, we use the notation $L[i, j]$

Algorithm 1: TIBOT(GO, L)

```

scan  $L$  to find all operations totally following  $L_r[m]$ 
and let them be  $L_{undo} = L[j, n]$ ;
if these operations exist then
    undo  $L_{undo}$  from  $L$ ;
    scan  $L$  and find the first operation  $O_i$ 
    such that  $TI(O_i) \geq TI(L_r[1])$ ;
    if no such operations exist then
        do  $L_r[1, m]$ ;
        append  $L_r[1, m]$  to end of  $L$ ;
    else
         $L_r[1, m] \leftarrow SLOT(L_r[1, m], L[i, j - 1])$ ;
        do  $L_r[1, m]$  in a sequel;
        append  $L_r[1, m]$  to end of  $L$ ;
    end if
     $L'_{undo} \leftarrow SLOT(L_{undo}, L_r[1, m])$ ;
    redo  $L'_{undo}$  in a sequel;
    append  $L'_{undo}$  to end of  $L$ ;
else
    scan  $L$  and find the first operation  $O_i$ 
    such that  $TI(O_i) \geq TI(L_r[1])$ ;
    if no such operations exist then
        do  $L_r[1, m]$ ;
    else
         $L_r[1, m] \leftarrow SLOT(L_r[1, m], L[i, n])$ ;
        do  $L_r[1, m]$  in a sequel;
        append  $L_r[1, m]$  to end of  $L$ ;
    end if
end if
end if

```

to denote a sublist of L ranging from O_i to O_j . The notation $L[i]$ denotes the i -th element O_i . We use $L_r[1, m]$ to represent the remote GO . The local history buffer HB is passed in for formal parameter L .

In the following we define the SLOT (Symmetric Linear Operation Transformation). SLOT implements symmetric inclusion transformation between two sequence of operations. As a result, the two sequences are updated so that each respectively includes the effects of each other.

Algorithm 2: $SLOT(L_1, L_2) : L'_1$

```

for( $i = 1; i \leq |L_1|; i++$ )
    for( $j = 1; j \leq |L_2|; j++$ )
         $SIT(L_1[i], L_2[j])$ 
    return  $L'_1 = L_1$ ;

```

The following SIT (Symmetric Inclusive Transformation) procedure is defined to inclusively transform two operations, O_1 and O_2 , against each other.

Algorithm 3: $SIT(O_1, O_2)$

```

 $O'_1 = IT(O_1, O_2)$ ;
 $O'_2 = IT(O_2, O_1)$ ;
 $O_1 = O'_1$ ;
 $O_2 = O'_2$ ;

```

The SLOT and SIT algorithms are adopted from [13] to achieve the following three goals in TIBOT: (1) Any remote group operation GO is transformed so that the effects of GO can be replayed correctly at current document state. (2) No operation is inserted in any GO and the ordering of operations in the GO is not changed. This way the precedence relation of all operations in any GO will not be changed. (3) Our approach achieves a linear time complexity. Compared with existing algorithms, assume that a remote group operation GO only includes one operation O_r . When O_r is synchronized, there are n operations in the history buffer, i.e., $HB = [O_1, \dots, O_n]$. According to the control algorithm, in the worst case, O_r needs to be transformed against n operations in HB . Hence the complexity of the TIBOT control algorithm is $O(n)$.

V. COMPARISON AND SIGNIFICANCE

Traditional consistency models (e.g., [1] [18]) generally adopt the read/write semantics and often ensure convergence by effectively serializing write operations. However, from the example shown in Fig. 1, we can show that serialized execution of write operations (in the forms of insert and delete) in group editors may cause undesirable effects on the user interfaces. Without loss of generality, assume the global total order in Fig. 1 is $O_1 < O_2 < O_3$. If the operations are executed in their original forms, then the states of both sites will converge to “bcd” instead of “bxd”, after all three operations are executed. This result is obviously inconsistent with the intention of O_3 which is to delete character ‘c’ instead of ‘x’. This problem in traditional concurrency control methods is referred to as intention violation in [16].

Compared to previous operational transformation algorithms, TIBOT has significant advantages in terms of simplicity, efficiency, and correctness. First, TIBOT is based on linear time intervals instead of state vectors. State vectors are more complicated to maintain and compare. They increase the size of synchronization messages and there are scalability problems, e.g., in supporting late joining of participating sites. By using time intervals, such problems are avoided in TIBOT. Secondly but more importantly, by imposing propagation and synchronization rules, TIBOT solved the dOPT puzzle and TP2 puzzle more elegantly than previous work.

A. Removing the dOPT puzzle

Several algorithms in the literature, GOTO [16], [15], Jupiter [10], and adOPTed [12], are known to have solved the dOPT puzzle. The Jupiter algorithm can actually be considered as a 2-dimensional version of the adOPTed algorithm. We only examine how GOTO and adOPTed solve the puzzle.

Based on TP1, there are usually multiple operation paths from one document state to another state. Hence we can always find a desired transformation and execution path so that the dOPT puzzle is removed.

To solve the dOPT puzzle, adOPTed records all necessary information related to different operation paths from an initial document state to current document state at each site. A N-dimensional history buffer is used for this purpose. When

scenarios of the dOPT puzzle occur, adOPTed simply chooses a proper operation path from the N-dimensional history buffer and performs a sequence of transformations along this path.

GOTO only keeps a history of operations that have been executed at each site. In other words, only one operations path is stored. So some useful information is lost. To solve the dOPT puzzle, GOTO always dynamically computes the necessary operation execution form in a given context by IT and ET, based on operations stored in the history buffer. Thus GOTO only needs a one-dimensional history buffer. Unfortunately, although theoretically IT and ET are reversible, that is, if $O'_1 = IT(O_1, O_2)$, then $O_1 = ET(O'_1, O_2)$, this reversibility cannot always be guaranteed in reality due to information loss in IT transformation [16].

Due to our earlier analysis, TIBOT avoids the dOPT scenarios. Therefore we do not need to keep additional information or dynamically compute such information. Then ET is removed and only a one-dimensional history buffer is necessary.

GOTO adopts a one-dimensional storage space. However, due to its need to dynamically computing the necessary operation execution forms, the time complexity of the GOTO control algorithm is $O(n^2)$. The complexity of adOPTed is $O(n)$, since necessary operation execution forms are stored in a N-dimensional history buffer. TIBOT achieves the same time complexity of $O(n)$ with the storage complexity significantly reduced, as compared to adOPTed, and the algorithm itself significantly simplified, as compared to GOTO.

B. Removing the TP2 puzzle

TP2 ensures that the correctness of operational transformation is not dependent on transformation path.

In figure 2, operations O_1 , O_2 , and O_3 are concurrent with each other. At site 2, O_2 has been executed before O_3 arrives. O_3 is transformed against O_2 to O'_3 . When O_1 arrives, it is transformed against O_2 and O'_3 in order. We have $O'_1 = IT(IT(O_1, O_2), O'_3)$.

At site 3, O_3 is executed first and then O_2 arrives. After transformation, $O'_2 = IT(O_2, O_3)$. When O_1 arrives, it is transformed against O_3 and O'_2 in order. We have $O''_1 = IT(IT(O_1, O_3), O'_2)$.

Based on TP1, we know that after both O_2 and O_3 are executed (although in different orders), the document states of site 2 and site 3 converge. Then how to execute O_1 at site 2 and 3? In other words, how to transform a remote operation against the same set of operations that have been executed in different orders at different sites? To ensure consistency, TP2 must be obeyed. Back to the above example, following TP2, if we ensure $O'_1 = O''_1$ or $IT(IT(O_1, O_2), O'_3) = IT(IT(O_1, O_3), O'_2)$, then executing O'_1 at both site 1 and site 2 will result in a consistent document state.

TP2 is required in most existing operational transformation algorithms [15] [4] [12], because an operation may have to be transformed against the same set of concurrent operations in different orders. Therefore, any OT control algorithm that can avoid transforming an operation against the same group of concurrent operations in different orders does not need TP2.

To prove why our model is free from TP2, we consider the following two aspects. (1) Transformation between operations carrying the same timestamp: Due to property 2, when a remote operation is executed locally, it always needs to be transformed against all operations carrying the same timestamp, if these operations exist. And this procedure is repeated at all involved sites. The undo/do/redo rule not only ensures that these operations are executed in the same order, but also that these transformations are processed in the total order, no matter where they happen. So in this case, it free TP2. After these operations are executed in the total order at all cooperating sites, the document states are consistent. (2) Transformation between remote operations and local operations carrying later timestamps: Due to property 1, such transformations only happen where these local operations are generated. Different from the case (1), these transformations are unique and site-specific so that we do not need to use the *undo/do/redo* scheme to achieve the same transformation order at different sites. Local operations can be executed immediately after they are generated. So, TP2 is also free in this case. Therefore, TP2 is free in our model. In fact, this also proves that our model is **convergent**.

Previous work, [16], [13], and [17], discussed how to free the TP2 puzzle. The approach proposed in [13] uses a centralized notification server to restrict the communication among client sites. In this way the otherwise arbitrary operation propagation in a pure replicated architecture is constrained by clients taking turns to propagate operations through the server. SOCT3 and SOCT4 [17] use a sequencer [11] to enforce a global total order that is free from TP2. GOT [16] ensures a total order by adopting a *undo/do/redo* scheme but is based on exclusion transformation (ET). As noted in [16], ET cannot always be implemented because of the information loss in IT. In addition, the use of ET is more complicated than IT. It increases the complexity of the control algorithm and requires extra storage spaces.

In contrast, our model still assumes a fully replicated architecture. Consistency problems in replicated systems are significantly harder than in centralized systems. Our result is thus interesting and significant on its own right. Our approach avoids relying on extra mechanisms such as the notification server in [13] and the sequencer in [17], which can become a single point of failure. By eliminating the use of ET, our algorithm is more efficient and simple than GOT [16].

Our recent work on SDT (state difference transformation) [8] also solved the TP2 puzzle. TIBOT and SDT have the following important differences: First, as all previous OT algorithms, SDT uses a vector clock approach for causality preservation. Second, SDT solves the TP2 problem by fixing problems in the original IT functions of [4] and avoiding the problems of ET as defined in [16]. By comparison, TIBOT solves the TP2 problem by fixing the control algorithm while keeping the original IT functions. Hence they are different approaches to solving the same problem. We plan to integrate these two approaches in one algorithm in future work.

VI. CORRECTNESS PROOF

Causality preservation and convergence are two conditions any consistency model must satisfy [3][16]. In this section we discuss how our model achieves these two properties. In fact, we have proved that our model is convergent in section 6.2. So in this section, we only prove causality preservation.

Let O be an operation generated at site i . That O is causally-ready for execution at site j only depends on rules we defined earlier, because these rules ensures the following conditions are satisfied: (1) all operations preceding O must be operations with earlier time interval or operations which is in same time interval and from the same site as O ; (2) when O is synchronized, all operations in time intervals earlier than O must have been synchronized; (3) when O is synchronized, all operations in the same group operation as O and generated before O must have been synchronized.

Proof: assume there exists an operation O' such that $O' \rightarrow O$ and $TI(O') > TI(O)$. If O' and O are generated from the same site, since $O' \rightarrow O$, according to the precedence relation definition, O' must have been generated before O . So $TI(O') \leq TI(O)$. This conclusion contradicts our assumption. Again, if O' and O come from different sites, according to the precedence relation definition, O must be generated after O' has been executed. The document context O based on includes the effect of O' . Meantime, according to propagation and synchronization rule, when O' is executed, the document context O' based on actually includes the effect of all operations with earlier timestamps. In other words, the document context O' based on includes the effect of O . These conclusions contradict each other. Therefore, we have proved that the first condition can be satisfied.

Then we prove the second and third conditions. These two condition are obviously satisfied, according to propagation and synchronization rules. Altogether these three conditions ensure that all operations preceding O have been executed at site j before O . Therefore causality is preserved in our model.

VII. CONCLUSION

This paper makes significant contributions as compared to related previous work. First, we use a linear time interval based logical clock for the same purpose of causality preservation as the more complex vector clock approach in existing operational transformation algorithms. This increases system scalability in terms of accommodating late comers in a dynamic collaboration environment. Second, we solve the dOPT puzzle with a one-dimensional history buffer (as compared to the N-dimensional storage in adOPTed [12]) and the time complexity the TIBOT control algorithm is $O(n)$ (as compared to $O(n^2)$ in GOTO [16][15]). Third, we solve the TP2 puzzle in a fully replicated architecture and without using ET (as compared to GOT [16]) or extra mechanisms (as compared to notification server in [13] and sequencer in [17]). The assumptions we made in this paper that lead to these results are reasonable in the target application domain of distributed interactive groupware systems.

We plan to extend this work by allowing for arbitrary time intervals at collaborating sites. Time intervals will be allowed to vary independently among different sites and to dynamically change during collaboration. With the extension, our algorithm will be able to support a continuous range of asynchronous and synchronous cooperative editing scenarios by allowing end users to simply set the time interval parameter. Further, by allowing for arbitrary intervals, the system will become more resilient to site and communication failures.

ACKNOWLEDGMENT

This work was supported in part by the National Science Foundation under CAREER award IIS-0133871 of Du Li. The authors can be contacted by email at lidu@cs.tamu.edu.

REFERENCES

- [1] B. Bhargava. Concurrency control in database systems. *IEEE Transactions on Knowledge and Data Engineering*, 11(1):3–16, 1999.
- [2] G. V. Cormack. A calculus for concurrent update. Technical Report CS-95-06, Dept. of Computer Science, University of Waterloo, Canada, 1995.
- [3] G. Coulouris, J. Dollimore, and T. Kindberg. *Distributed Systems Concepts and Design*. Addison-Wesley, 2001.
- [4] C. A. Ellis and S. J. Gibbs. Concurrency control in groupware systems. In *ACM SIGMOD'89 Proceedings*, pages 399–407, Portland Oregon, 1989.
- [5] S. Greenberg and D. Marwood. Real-time groupware as a distributed system: Concurrency control and its effect on the interface. In *ACM CSCW'94 Proceedings*, pages 207–217, Chapel Hill, NC, Oct. 1994.
- [6] M. J. Knister and A. Prakash. A framework for undoing actions in collaborative systems. *ACM Transactions on Computer-Human Interaction*, 1(4):295–330, Dec. 1994.
- [7] L. Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, July 1978.
- [8] D. Li and R. Li. Ensuring content and intention consistency in real-time group editors. In *Proceedings of the 24th IEEE International Conference on Distributed Computing Systems (ICDCS'04)*, pages 748–755, Mar. 2004.
- [9] D. Li, C. Sun, L. Zhou, and R. R. Muntz. Operation propagation in real-time group editors. *IEEE Multimedia Special Issue on Multimedia Computer Supported Cooperative Work*, 7(4):55–61, 2000.
- [10] D. A. Nichols, P. Curtis, M. Dixon, and J. Lamping. High-latency, low-bandwidth windowing in the jupiter collaboration system. In *ACM UIST'95 Proceedings*, Nov. 1995.
- [11] D. P. Reed and R. K. Kanodia. Synchronization with eventcounts and sequencers. *Communications of the ACM*, 22(2):115–123, Feb. 1979.
- [12] M. Ressel, D. Nitsche-Ruhland, and R. Gunzenhauser. An integrating, transformation-oriented approach to concurrency control and undo in group editors. In *ACM CSCW'96 Proceedings*, pages 288–297, Nov. 1996.
- [13] H. Shen and C. Sun. Flexible notification for collaborative systems. In *ACM CSCW'02*, pages 77–86, Nov. 2002.
- [14] C. Sun. Optional and responsive fine-grain locking in Internet-based collaborative systems. *IEEE Transactions on Parallel and Distributed Systems*, 13(9):994–1008, Sept. 2002.
- [15] C. Sun and C. Ellis. Operational transformation in real-time group editors: issues, algorithms, and achievements. In *ACM CSCW'98*, pages 59–68, Dec. 1998.
- [16] C. Sun, X. Jia, Y. Zhang, Y. Yang, and D. Chen. Achieving convergence, causality-preservation, and intention-preservation in real-time cooperative editing systems. *ACM Transactions on Computer-Human Interaction*, 5(1):63–108, Mar. 1998.
- [17] N. Vidot, M. Cart, J. Ferrie, and M. Suleiman. Copies convergence in a distributed realtime collaborative environment. In *ACM CSCW'00*, pages 171–180, Dec. 2000.
- [18] H. Yu and A. Vahdat. Design and evaluation of a conit-based continuous consistency model for replicated services. *ACM Transactions on Computer Systems*, 20(3):239–282, Aug. 2002.